B.12 — Rigenerazione del magic-link impresa

Guida all'implementazione del flusso B.12

Progetto ICMVerbali

2026-05-15

Introduzione

Questo documento estende B11-firma-impresa descrivendo la sotto-fase **B.12**: la possibilità per il CSE di **rigenerare il magic-link** dell'Impresa Appaltatrice dalla detail view di un verbale già firmato.

In B.11 abbiamo introdotto il flusso `FirmatoCse → FirmatoImpresa` via magic-link a uso singolo, con scadenza 48h, generato atomicamente alla firma del CSE. B.12 chiude un buco operativo: cosa fare quando il link va perso o non viene inoltrato in tempo.

Il documento assume la lettura di B.11 e di `docs/01-design.md`. Qui ci concentriamo su **cosa è cambiato rispetto a B.11** e sulle **scelte di design** della rigenerazione.

---

1. Il problema da risolvere

In B.11 il magic-link veniva generato una sola volta, all'atto della firma CSE, e mostrato in un dialog post-firma con un bottone "Copia link". Funzionale, ma fragile: una volta chiuso il dialog, **non c'era più modo di recuperare quel link**. I casi tipici emersi nell'uso reale:

- Il CSE dimentica di inoltrare il link e poi non sa più dove ripescarlo.
- L'Impresa cancella per errore il messaggio col link.
- Le 48h passano senza che nessuno apra il link, e ora serve riemetterlo.

Prima di B.12 l'unica via d'uscita era intervenire a mano sul DB. B.12 risolve dando al CSE un'azione esplicita di rigenerazione direttamente dalla **detail view del verbale** (`/verbali/{id}`), accessibile dalla Home come già introdotto in B.11.

---

2. Cosa abbiamo costruito in B.12

Le modifiche, tutte additive rispetto a B.11:

1. **Migration `003_AddRevocatoUtcToFirmaToken.sql`**: aggiunge la colonna `RevocatoUtc datetime2(0) NULL` alla tabella `FirmaToken`, per rappresentare la revoca esplicita di un token.
2. **`FirmaTokenRepository`** estesa con due nuovi metodi: `GetUltimoAttivoAsync` (lookup del token utilizzabile più recente) e `RigeneraAsync` (transazione che revoca i token attivi e ne inserisce uno nuovo).
3. **`FirmaTokenManager`** estesa con `RigeneraTokenAsync` (orchestrazione con pre-check di stato) e `GetLinkAttivoAsync` (passthrough).
4. **`ValidaTokenAsync`** aggiunge un nuovo motivo `Revocato` (precede `Scaduto` e `GiaUsato` nel controllo).

5. **`VerbaleRepository.SqlMarkTokenUsato`** filtra anche `RevocatoUtc IS NULL`, come difesa contro firme con link vecchi rimasti aperti.
6. **`VerbaleDetail.razor`** mostra un nuovo blocco "Link firma Impresa" se il verbale è in `FirmatoCse`, con bottoni Copia/Rigenera o "Genera nuovo link".
7. Nuovo evento di audit `EventoAuditTipo.RigenerazioneToken = 4`.

---

3. Flusso end-to-end

Dalla prospettiva del CSE.

Il CSE rigenera un link

1. Dalla Home apre un verbale in stato `FirmatoCse` → atterra su `/verbali/{id}`.
2. Sotto la sezione 9 (foto), prima della 10 (firme), trova il blocco **"Link firma Impresa"**.
3. Se esiste un link ancora attivo, lo vede in chiaro in un campo read-only con la data/ora di scadenza in formato locale, più due bottoni: **Copia link** e **Rigenera link**.
4. Click su **Rigenera link** → conferma `MessageBox` ("Il link attualmente attivo verrà invalidato. Eventuali copie già inviate all'Impresa smetteranno di funzionare. Procedere?").
5. Conferma → backend rigenera in una transazione DB. Il vecchio token diventa `Revocato`, il nuovo è `Attivo`.
6. Si apre il `LinkImpresaDialog` (lo stesso usato post-firma CSE in B.11) con il nuovo URL pronto da copiare.

Variante: nessun link attivo

Se tutti i token del verbale sono scaduti, usati o revocati, il blocco mostra un alert giallo "Nessun link attivo" e un solo bottone **Genera nuovo link**, che salta la conferma (non c'è niente da invalidare).

L'Impresa apre un link revocato

L'Impresa che apre un link nel frattempo revocato vede ora un messaggio dedicato — non "scaduto", che la confonderebbe — ma **"Link sostituito: il CSE ha generato un nuovo link per questo verbale. Richiedi quello aggiornato."** Vedi §6.

---

4. Modello dati: la colonna `RevocatoUtc`

```
ALTER TABLE dbo.FirmaToken ADD RevocatoUtc datetime2(0) NULL;
```

Una sola colonna nullable, nessun nuovo indice. Il rationale per non riusare campi esistenti:

- **Non riusare `UsatoUtc`** per marcare la revoca: i due eventi sono semanticamente diversi. *Usato* significa "una firma è stata applicata"; *revocato* significa "il CSE ha sostituito il link, nessuna firma è arrivata". Mescolarli renderebbe impossibile contare quante firme magic-link sono effettivamente avvenute.
- **Non cancellare la riga**: l'audit trail del verbale richiede di poter ricostruire chi ha rigenerato, quando e quanti link sono stati emessi prima del consumo finale.

Da B.12 in poi, lo stato logico di un `FirmaToken` è una funzione di tre colonne (`UsatoUtc`, `RevocatoUtc`, `ScadenzaUtc` rispetto a `now`) e ricade in uno di quattro casi mutuamente esclusivi:

Stato	UsatoUtc	RevocatoUtc	ScadenzaUtc	Significato
Attivo	NULL	NULL	> now	Utilizzabile per firmare
Usato	valued	NULL	qualsiasi	Firma applicata (consumato)
Revocato	NULL	valued	qualsiasi	Sostituito dal CSE con un nuovo token
Scaduto	NULL	NULL	<= now	48h passate senza utilizzo né rigenera

L'ordine di precedenza nei controlli è: **Revocato → Usato → Scaduto** (vedi §6).

---

5. Le decisioni di design ratificate

Tre scelte centrali confermate con l'utente in apertura della sessione:

5.1 Revoca esplicita vs "ultimo vince"

L'alternativa a introdurre la colonna era lasciar convivere i vecchi link col nuovo, finché ognuno non scade per conto proprio. Più snello (zero migration), ma con due problemi:

- Il CSE che rigenera "per sicurezza" dopo aver già condiviso un link finirebbe per avere due link entrambi validi in giro.
- L'audit non distinguerebbe mai tra "scaduto inutilizzato" e "sostituito attivamente".

La revoca esplicita rende il modello mentale univoco: **solo l'ultimo link funziona**. Costo di una colonna e di una UPDATE in più nella transazione.

5.2 UI: link visibile + due bottoni vs "solo bottone"

L'alternativa era nascondere sempre il link e mostrare solo "Rigenera/Genera link" che apre il dialog. Più semplice, ma costringe il CSE a un click anche solo per copiare un link già valido — peggiore in termini di UX per il caso d'uso più comune ("ho perso il messaggio, mi serve il link che ho già emesso").

Mostrare il link attivo direttamente nella detail view, con la copia separata dalla rigenerazione, riduce gli errori: chi vuole solo recuperare il link già valido **non deve invalidarlo**.

5.3 Chi può rigenerare

Tutti gli utenti autenticati che possono accedere alla detail view. Nessuna restrizione aggiuntiva: la rigenerazione è un'operazione di servizio, l'audit ne traccia l'autore. Una restrizione più stretta (solo l'utente che ha firmato CSE) sarebbe stata ortogonale al problema senza beneficio chiaro.

---

6. La gerarchia dei motivi in `ValidaTokenAsync`

`FirmaTokenInvalidoMotivo` ora ha quattro valori:

```
public enum FirmaTokenInvalidoMotivo : byte
{
    NonTrovato = 0,
    Scaduto    = 1,
```

```
    GiaUsato    = 2,
    Revocato    = 3,
}
```

L'ordine dei controlli in `ValidaTokenAsync` non è arbitrario. È: prima esistenza del token (`NonTrovato`), poi `Revocato`, poi `GiaUsato`, infine `Scaduto`.

La scelta di mettere **Revocato prima di tutto** (eccetto NonTrovato) è UX-driven. Considera questo scenario:

> Il CSE genera un link alle 10:00. Alle 11:00 lo rigenera. Alle 13:00 (link non scaduto, scadenza = +48h) l'Impresa apre il vecchio link.

Senza la priorità su `Revocato`, l'Impresa riceverebbe il messaggio del primo motivo applicabile in ordine "fisico" — ad esempio "Scaduto" se per ipotesi le date di scadenza fossero passate, o uno generico. Con la priorità corretta, l'Impresa legge esattamente quel che è successo: **"Il CSE ha generato un nuovo link per questo verbale. Richiedi quello aggiornato."** È l'unico messaggio che le dà l'azione corretta da intraprendere.

---

7. Le difese contro le race condition

Un'azione che invalida link in giro per il mondo apre alcune finestre temporali su cui ragionare.

7.1 Doppia tab del CSE che rigenera

Caso: il CSE apre la detail view in due tab e clicca "Rigenera" in entrambe a distanza di un secondo. Cosa succede?

- Tab A: `RigeneraAsync` → revoca tutti gli attivi (zero righe da revocare a parte il token originale di B.11), inserisce token T_A.
- Tab B: `RigeneraAsync` → revoca tutti gli attivi (revoca T_A!), inserisce token T_B.

Risultato: `T_A` viene revocato prima di poter essere usato. Solo `T_B` resta attivo. Comportamento corretto: rigenerare due volte di seguito è equivalente a rigenerare una volta sola con il risultato dell'ultima.

7.2 Impresa apre il link mentre il CSE rigenera

Caso: l'Impresa ha appena aperto la pagina `/firma-impresa/{T_old}` e sta disegnando la firma; nel frattempo il CSE rigenera, `T_old` diventa revocato.

- L'Impresa clicca "Conferma firma" → `VerbaleManager.FirmaImpresaAsync(T_old)`.
- Il manager chiama `ValidaTokenAsync(T_old)` → ora ritorna `Revocato`, eccezione tipizzata, niente DB write.

Anche se in qualche modo il check del manager venisse aggirato, c'è una **seconda difesa** lato repository: lo `SqlMarkTokenUsato` ora filtra anche `RevocatoUtc IS NULL`:

```sql
UPDATE dbo.FirmaToken
SET UsatoUtc = SYSUTCDATETIME()
WHERE Id = @Id AND UsatoUtc IS NULL AND RevocatoUtc IS NULL;
```

Se la riga non viene aggiornata (zero righe toccate), `FirmaImpresaAsync` lancia `InvalidOperationException` e la transazione rollbacka. Lo stato del verbale resta `FirmatoCse` e nessuna firma viene erroneamente registrata.

Questa è la stessa difesa “0 rows = race lost” che B.11 usava per il caso “due tab dell’Impresa firmano insieme”; B.12 ne estende la copertura includendo la revoca.

7.3 Atomicità della rigenerazione

`RigeneraAsync` opera in transazione: UPDATE dei token attivi → INSERT del nuovo → INSERT in `VerbaleAudit`. Se uno qualsiasi degli step fallisce, lo stato del DB resta com’era. Niente token “appesi” senza audit, niente revoche orfane.

Non c’è invece un lock sul `Verbale` durante la rigenerazione: contendere con `FirmaImpresaAsync` non porterebbe valore (il check sullo stato lo fa il manager prima della transazione, e la difesa su `SqlMarkTokenUsato` chiude la finestra residua).

---

8. Audit

Un nuovo `EventoAuditTipo`:

```
public enum EventoAuditTipo : byte
{
    Creazione          = 0,
    TransizioneStato   = 1,
    Eliminazione       = 2,
    Firma              = 3,
    RigenerazioneToken = 4,  // B.12
}
```

La riga di audit inserita da `RigeneraAsync` riporta:

- `UtenteId` = utente loggato che ha cliccato “Rigenera” (preso via `AuthenticationStateProvider` lato componente, propagato al manager).
- `EventoTipo` = `RigenerazioneToken`.
- `Note` = `"Rigenerato magic-link impresa (nuovo TokenId {guid})."` — il TokenId del nuovo token, mai il `Token` GUID esposto nell’URL (per non lasciarlo nei log).

Esempio di ricostruzione audit di un verbale che ha richiesto due rigenerazioni prima della firma impresa:

```
1. Creazione               (CSE)        — bozza creata
2. Firma                   (CSE)        — firma CSE applicata, T1 emesso
3. RigenerazioneToken      (CSE)        — T1 revocato, T2 emesso
4. RigenerazioneToken      (CSE)        — T2 revocato, T3 emesso
5. Firma                   (CSE proxy)  — firma Impresa via T3, T3 marcato usato
```

(Per la nota su `UtenteId = CompilatoDaUtenteId` nella firma impresa, vedi B.11 §7.)

---

9. Cambiamenti riassuntivi per layer

9.1 Database

Una sola colonna nullable in `FirmaToken`. Migration idempotente nel pattern già usato per le precedenti.

9.2 Repository (`FirmaTokenRepository`)

Due nuovi metodi pubblici:

```
Task<FirmaToken?> GetUltimoAttivoAsync(Guid verbaleId, CancellationToken ct);
Task RigeneraAsync(Guid verbaleId, FirmaTokenInputs nuovoToken, Guid utenteId, CancellationToken ct);
```

`GetUltimoAttivoAsync` è una `SELECT TOP 1 ... ORDER BY CreatedAt DESC` con i tre filtri di stato attivo. `RigeneraAsync` è la transazione descritta in §7.3. Entrambi i metodi sono parte di `IFirmaTokenRepository`.

In `VerbaleRepository`, lo `SqlMarkTokenUsato` è stato modificato per includere il filtro su `RevocatoUtc IS NULL` (vedi §7.2). È l'unica modifica al `VerbaleRepository`, tutto il resto della classe è invariato.

9.3 Manager (`FirmaTokenManager`)

Tre nuovi membri esposti su `IFirmaTokenManager`:

```
Task<FirmaToken?> GetLinkAttivoAsync(Guid verbaleId, CancellationToken ct);
Task<Guid> RigeneraTokenAsync(Guid verbaleId, Guid utenteId, CancellationToken ct);
// ValidaTokenAsync invariato come signature, esteso col nuovo motivo Revocato
```

Il manager ora dipende anche da `IVerbaleRepository` (serve per il pre-check di stato `FirmatoCse` in `RigeneraTokenAsync`). Allineato a `VerbaleManager.FirmaImpresaAsync` che fa il pre-check analogo.

`RigeneraTokenAsync` lancia `InvalidOperationException` con messaggio esplicito se lo stato del verbale è diverso da `FirmatoCse`: rigenerare su `Bozza` non ha senso (il token non esiste ancora), su `FirmatoImpresa` neppure (la firma è già stata applicata, il token è stato consumato).

9.4 UI (`VerbaleDetail.razor`)

Il blocco "Link firma Impresa" appare solo quando `_verbale.Stato == StatoVerbale.FirmatoCse`. Marcato `no-print` (non finisce nelle stampe).

Logica di rendering:

- `OnInitializedAsync` carica `_linkAttivo = await FirmaTokenManager.GetLinkAttivoAsync(...)` insieme agli altri dati.
- Se `_linkAttivo != null`: campo `MudTextField` ReadOnly con il URL completo (`{Nav.BaseUri}firma-impresa/{token}`), data/ora scadenza in `ToLocalTime`, bottoni Copia + Rigenera.
- Se `_linkAttivo == null`: alert giallo + un solo bottone "Genera nuovo link".

`HandleCopyAsync` riusa il modulo JS `/js/clipboard.js` introdotto in B.11 (con fallback `document.execCommand` per contesti http LAN — vedi memoria `feedback_ngrok_lan_test.md` e B.11 §10 bug fix).

`HandleRigeneraAsync` chiede conferma `MessageBox` solo quando esiste già un link attivo (rigenerare invalida l'esistente, ed è un'azione che merita un click consapevole). Estrae l'utente loggato via `AuthenticationStateProvider` (stesso pattern di `SignaturePadDialog`), chiama `FirmaTokenManager.RigeneraTokenAsync`, ricarica `_linkAttivo` e apre `LinkImpresaDialog` col nuovo URL.

9.5 Pagina pubblica (`FirmaImpresa.razor`)

Solo due righe modificate: aggiunta dei mapping per `FirmaTokenInvalidoMotivo.Revocato` in `TokenErrorTitle` e `TokenErrorMessage`, per produrre il messaggio "Link sostituito" descritto in §3.

10. Test

10 test nuovi, tutti verdi (totale **50/50**, da 40/40 al checkpoint B.11).

10.1 Unit (`FirmaTokenManagerTests`, niente DB)

- `ValidaTokenAsync_token_revocato_lancia_Revocato`: verifica la priorità della revoca (un token revocato ma non scaduto e non usato lancia `Revocato`, non altri motivi).
- `RigeneraTokenAsync_su_FirmatoCse_genera_nuovo_token_e_chiama_repo`: verifica che il manager pre-calcoli il seed e passi al repo i parametri attesi (verbaleId, utenteId, token GUID inedito).
- `RigeneraTokenAsync_su_stato_diverso_da_FirmatoCse_throw` (Theory ×3 stati: Bozza, FirmatoImpresa, Chiuso): in tutti i casi `InvalidOperationException` e nessuna chiamata al repo.
- `RigeneraTokenAsync_verbale_inesistente_throw`: pre-check fallisce sul lookup del verbale.
- `GetLinkAttivoAsync_passthrough_al_repository`: il manager non fa logica aggiuntiva oltre al passthrough.

10.2 Integrazione (`VerbaleRepositoryTests`, contro SQL Express)

- `RigeneraAsync_revoca_token_attivi_e_inserisce_nuovo_con_audit`: dopo firma CSE + rigenerazione, il primo token risulta revocato (UsatoUtc null, RevocatoUtc valorizzato), il nuovo è attivo, l'audit contiene la riga `RigenerazioneToken`.
- `GetUltimoAttivoAsync_ignora_usati_revocati_scaduti_e_torna_il_piu_recente`: scenario realistico in 5 step (no token → firma CSE → rigenera → forza scadenza → rigenera ancora) verificando che la query torni sempre il token corretto.
- `FirmaImpresaAsync_token_revocato_lancia_InvalidOperationException`: verifica end-to-end della difesa su `SqlMarkTokenUsato`. Si firma CSE, si rigenera, si tenta di firmare impresa col token revocato → la transazione rollbacka, lo stato del verbale resta `FirmatoCse`.

---

11. Cosa NON è stato fatto in B.12 (di proposito)

- **Notifica automatica all'Impresa** quando il CSE rigenera (email/SMS): la rigenerazione resta un'azione manuale che il CSE accompagna inviando il nuovo link via canale a sua scelta. La notifica automatica entra in scena solo quando avremo un `IEmailService` configurato (rimandato).
- **Limite al numero di rigenerazioni per verbale**: nessun rate-limit, nessun contatore. Se in produzione emergessero abusi (improbabile, è un'azione interna a operatori autenticati), si introdurrà.
- **Visualizzazione storica dei token** nella detail view: la detail view mostra solo l'ultimo attivo. La cronologia completa (utile per audit e debug) resta accessibile via `IFirmaTokenRepository.GetByVerbaleAsync`, ma non è esposta in UI.
- **UI per rigenerare anche su `Chiuso`**: per ora `FirmatoCse` è l'unico stato in cui la rigenerazione ha senso. Se mai dovessimo riaprire un verbale chiuso per una correzione, sarà una funzionalità separata.

---

12. Pattern riusabili emersi in B.12

- **Il pattern "stato logico derivato da N colonne nullable"** (qui: Attivo/Usato/Revocato/Scaduto sintetizzati da tre colonne) è applicabile ovunque si voglia evitare un enum di stato esplicito che dovrebbe

essere mantenuto in sync. Il rovescio è che le query devono replicare i predicati: vale la pena solo quando gli stati sono pochi e ortogonali.

- **La doppia difesa "manager pre-check + repository sentinel"** (qui: `ValidaTokenAsync` + `WHERE RevocatoUtc IS NULL` su `MarkTokenUsato`) è il pattern che rende la transizione robusta a TOCTOU. Il pre-check serve per il messaggio user-friendly, la sentinel serve per la correttezza. Da ricordare per ogni futura UPDATE che dipende da uno stato letto in precedenza.

- **L'ordine dei motivi in un'eccezione tipizzata è UX**: cambia il messaggio che l'utente legge. La regola: il motivo che rappresenta meglio "cosa è successo davvero" e che meglio guida l'azione successiva va per primo.